CIS-11 Project Documentation

Test Score Calculator

Team Name: Team Stack

Team Members: Joseph Rodriguez & Lucas Vida

Project Name: LC-3 Test Score Calculator

Date: May 24, 2026

Advisor: Kasey Nguyen, PhD

Part I - Application Overview

Objectives

The objective of this project is to create an LC-3 assembly language program that reads five test scores, stores the scores in memory, calculates the minimum score, maximum score, and integer average, and displays each calculated result with its matching letter grade. The final LC-3 code uses the prompt "Enter Score: " for each score and stores the five numeric values in the SCORES array.

The program demonstrates memory addressing, array storage, pointer use, branching, loops, subroutines, stack-based register save/restore, ASCII-to-integer conversion, integer-to-ASCII output, and LC-3 TRAP services. The code uses R6 as the stack pointer, R1 as the score-array pointer during input and processing, R2 as a counter or running total depending on the routine, and R4 as the running sum during statistics calculation.

- Read and store exactly five test scores using the SCORES array.
- Calculate and display the minimum score, maximum score, and integer average.
- Display a letter grade for the minimum, maximum, and average values using the scale A, B, C,
- D, and F.
- Use LC-3 subroutines including READ_SCORE, CALC_STATS, PRINT_RESULTS,
- PRINT_NUM, and PRINT_GRADE.
- Use stack operations with R6 to save and restore registers
- inside subroutines.

Business Process

The current process for calculating grades is manual. A user writes down five scores, compares the values to find the highest and lowest scores, adds the scores, divides the total by five, and then checks the resulting values against a grade scale. This can lead to arithmetic mistakes or inconsistent grading which is a big nightmare if a student where to get an incorrect letter grade.

The future process using this program is simpler. The user runs the LC-3 program, enters five scores through the console, and the program stores the scores, calculates the minimum, maximum, and average, converts numeric values for output, and displays the results with their letter grades. The application is intended for a classroom or lab environment where students practice LC-3 input, output, arithmetic, branching, memory, and subroutines.

User Roles and Responsibilities

User Role	Responsibilities	Use of System
Student/User	Enter five test scores, review the displayed output, and verify that the results are reasonable.	Runs the program in the LC-3 simulator and inputs scores when prompted.
Programmer: Lucas Vida	Code, debug, test, and verify the LC-3 assembly program.	Implements the final source code, including input, array storage, calculation, output, grade classification, and register save/restore behavior.
Documenter: Joseph Rodriguez	Prepare, revise, and align the project documentation with the final source code.	Documents the program overview, requirements, memory layout, subroutines, pseudocode, and project coverage.
Business: Professor Nguyen	Evaluate whether the project satisfies CIS-11 project requirements.	Reviews the documentation, final source code, comments, test behavior, and console output.

Production Rollout Considerations

The program will be submitted as an LC-3 assembly source file and a project documentation document. Lucas completed the coding, debugging, and testing in the LC-3 simulator, while Joseph completed the documentation from project idea to the final code. The only required environment is an LC-3 simulator that supports standard TRAP routines such as GETC, OUT, PUTS, and HALT.

The final code includes a test input comment of 48, 87, 96, 79, and 61. Additional test data should include boundary values such as 0, 59, 60, 69, 70, 79, 80, 89, 90, and 100. The READ_SCORE routine accepts valid digit characters, ignores non-digit characters, stops input when Enter is pressed, and clamps any value above 100 down to 100. It does not display a separate error message or re-prompt when a value exceeds 100.

Terminology

Term	Definition
LC-3	Little Computer 3, the assembly language and simulator used in CIS-11.
TRAP routine	A system call/service used for input, output, or stopping the program. This code uses GETC, OUT, PUTS, and HALT.
ASCII conversion	The process of converting typed digit characters into numeric values and converting numeric values back into printable characters.
Array	A group of memory locations used to store the five test scores. In this program, the array label is SCORES.
Pointer	A register or memory variable used to track the current array address. In this program, R1 points into SCORES, and SPTR stores the current pointer during input.
Stack	Memory used to save and restore register values. This program initializes R6 from STACK and adjusts R6 inside subroutines.
Subroutine	A reusable block of LC-3 code called with JSR and returned from with RET.
Integer average	An average calculated using integer division. This program divides the sum by five using repeated subtraction and stores the quotient in AVGV.
Clamping	Limiting a value to a maximum. In READ_SCORE, values greater than 100 are changed to 100.

Part II - Functional Requirements

Statement of Functionality

1. Initialize the program at .ORIG X3000 and set up the stack pointer by loading R6 from STACK, which

2. contains xFDFF.
3. Load the starting address of the SCORES array into R1 and store the current pointer in SPTR.
4. Use CTR and FIVE to control an input loop that reads exactly five scores.
5. Prompt the user with the string "Enter Score: " before each score is entered.
6. Use the READ_SCORE subroutine to read keyboard input with GETC, accept digit characters, echo valid digits using OUT, stop at Enter, convert ASCII digits to an integer value, and clamp values above 100 to 100.
7. Store each completed score into the SCORES array with STR and advance the score pointer after each stored value.
8. Use CALC_STATS to walk through the SCORES array, calculate MINV, MAXV, and AVGV, and use R4 as the running sum during processing.
9. Compute the average using repeated subtraction by five, storing the integer quotient in AVGV.
10. Use PRINT_RESULTS to display the minimum, maximum, and average values with output labels.
11. Use PRINT_NUM to convert an integer value into ASCII digits and print the digits in the correct order.
12. Use PRINT_GRADE to print a letter grade for the value currently in R0 using thresholds T90, T80, T70, and T60.
13. Classify grades using the scale 90-100 = A, 80-89 = B, 70-79 = C, 60-69 = D, and 0-59 = F.
14. Use stack-based save/restore logic in subroutines by decrementing R6, storing registers with STR, restoring with LDR, and incrementing R6 before RET.
15. Use LC-3 system calls GETC, OUT, PUTS, and HALT.

Scope

The project scope is limited to a console-based LC-3 program that processes five test scores. The program does not include file storage, graphics, decimal averages, network access, a user-account system, or advanced error reporting.

In scope: five-score input, SCORES array storage, min/max/integer-average calculation, letter grade display for the displayed results, input/output messages, subroutines, stack-based save/restore, pointer use, and ASCII conversion.

Out of scope: graphical interface, permanent saved records, floating-point decimal averages, network access, and full invalid-input error handling. The final code skips non-digit characters and clamps values above 100 rather than displaying an error and re-prompting.

Performance

The program completes its calculations immediately after the fifth score is entered. Since exactly five scores are processed, the input and calculation loops have small fixed limits. The maximum

stored score is clamped to 100, so the intended total remains within the safe range for this assignment.

Prompt and accept five scores during one program run.
Store all five scores in the SCORES array.
Calculate and store MINV, MAXV, and AVGV before output.
Display the calculated results and corresponding letter grades before halting.
Clamp values greater than 100 down to 100 in READ_SCORE.

Usability

The console output is simple and labeled. The program uses the same prompt, "Enter Score: ", for each of the five inputs instead of numbering the prompts. After all scores are entered, the program displays Minimum, Maximum, and Average results. Each displayed result is followed by a Grade label and a single letter grade.

Example console interaction using the code comment test input:

```
Enter Score: 48
Enter Score: 87
Enter Score: 96
Enter Score: 79
Enter Score: 61

Minimum:  48  Grade:  F
Maximum:  96  Grade:  A
Average:  74  Grade:  C
```

Documenting Requests for Enhancements

Date	Enhancement	Requested By	Priority	Status / Notes
May 24, 2026	Select the Test Score Calculator project.	Team Stack	High	Completed.
May 25, 2026	Begin project documentation.	Joseph Rodriguez	High	Completed.
May 26, 2026	Create pseudocode and identify required subroutines.	Team Stack	High	Completed.
May 27, 2026	Create first coding trial.	Lucas Vida	High	Completed.

May 28, 2026	Fix compiling and logic errors.	Lucas Vida	High	Completed.
June 1, 2026	Revise documentation and align with final program structure.	Joseph Rodriguez	High	Completed.
June 3, 2026	Finalize code, debugging, and testing.	Lucas Vida	High	Completed.
June 4, 2026	Finalize documentation	Joseph Rodriguez	High	Completed.

Part III - Appendices

Appendix A - Project Requirement Coverage

Requirement	How the Program Meets It	Example Labels / Instructions
Appropriate addresses and storage	Uses .ORIG for the program start and .FILL, .BLKW, and .STRINGZ for constants, variables, arrays, and messages.	.ORIG X3000, STACK .FILL xFDFF, SCORES .BLKW 5, PROMPT .STRINGZ
Display min/max/average/grade	PRINT_RESULTS prints Minimum, Maximum, and Average. Each printed value is followed by Grade and a letter.	PRINT_RESULTS, MSG_MIN, MSG_MAX, MSG_AVG, MSG_GRADE, PRINT_NUM, PRINT_GRADE
Labels and comments	Uses descriptive labels for loops, routines, constants, variables, arrays, and messages.	MAIN, INPUT_LOOP, READ_SCORE, CALC_STATS, PRINT_RESULTS, PRINT_NUM, PRINT_GRADE
Arithmetic, data movement, and conditional instructions	Uses ADD, AND, LD, LEA, LDR, STR, ST, NOT, and BR instructions for arithmetic, storage, comparisons, and loops.	ADD for totals and subtraction, LDR/STR for array access, BRn/BRz/BRnp/BRnzp for flow control
Two or more subroutines	Divides program into reusable routines for input, calculations,	READ_SCORE, CALC_STATS,

	output, number printing, and grade printing.	PRINT_RESULTS, PRINT_NUM, PRINT_GRADE
Branching and iteration	Uses an input loop for five scores, a calculation loop for array processing, repeated subtraction loops, and grade threshold branches.	INPUT_LOOP, CS_LOOP, CS_DIV, PN_DIV, PG_B, PG_C, PG_D, PG_F
Overflow/storage handling	Stores five values in SCORES and clamps input values above 100 to 100.	SCORES .BLKW 5, HUNDRED .FILL #100, NEG100 .FILL #-100, RS_CLAMP
Stack save/restore	Uses R6 as the stack pointer. Subroutines push registers by decrementing R6 and using STR, then restore registers with LDR and increment R6.	LD R6, STACK; ADD R6, R6, #-1; STR R7, R6, #0; LDR R7, R6, #0
Pointer use	Uses R1 as a pointer into SCORES and stores the current input pointer in SPTR.	LEA R1, SCORES; ST R1, SPTR; STR R0, R1, #0; ADD R1, R1, #1
ASCII conversion	READ_SCORE subtracts 48 from typed characters. PRINT_NUM adds ASCII0 to digit values before output.	NEG48 .FILL #-48, ASCII0 .FILL x0030
System call directives/services	Uses LC-3 TRAP services for keyboard input, character output, string output, and program halt.	GETC, OUT, PUTS, HALT

Appendix B - Pseudocode

```
BEGIN PROGRAM
    R6 <- STACK
    R1 <- address of SCORES
    SPTR <- R1
    CTR <- 5

    WHILE CTR is not zero
        PRINT PROMPT using PUTS
        CALL READ_SCORE
        R1 <- SPTR
```

```
        STORE R0 into SCORES through R1
        R1 <- R1 + 1
        SPTR <- R1
        CTR <- CTR - 1
    END WHILE

    CALL CALC_STATS
    CALL PRINT_RESULTS
    HALT
END PROGRAM

READ_SCORE
    Save R7, R2, R3, and R4 on the stack using R6
    R2 <- 0 running total
    LOOP
        GETC character into R0
        IF character is Enter THEN go to clamp step
        Convert character to digit by subtracting 48
        IF character is not 0 through 9 THEN ignore it and continue loop
        OUT valid digit
        R2 <- (R2 * 10) + digit
        Repeat loop
    CLAMP
        Print newline
        IF R2 > 100 THEN R2 <- 100
        R0 <- R2
    Restore registers and RET

CALC_STATS
    Save registers on stack
    Load first score from SCORES
    MINV <- first score
    MAXV <- first score
    R4 <- first score as running sum
    Process remaining four scores
        Load current score through pointer R1
        IF current score < MINV THEN MINV <- current score
        IF current score > MAXV THEN MAXV <- current score
        Add current score to running sum R4
        Move pointer to next score
    Divide running sum by 5 using repeated subtraction
    AVGV <- quotient
    Restore registers and RET

PRINT_RESULTS
    Print MSG_MIN, MINV, MSG_GRADE, and grade for MINV
    Print MSG_MAX, MAXV, MSG_GRADE, and grade for MAXV
    Print MSG_AVG, AVGV, MSG_GRADE, and grade for AVGV
    RET
```

```
PRINT_NUM
    Convert number in R0 to ASCII digits
    Use the stack to reverse digit order
    OUT each digit
    RET

PRINT_GRADE
    IF R0 >= 90 print A
    ELSE IF R0 >= 80 print B
    ELSE IF R0 >= 70 print C
    ELSE IF R0 >= 60 print D
    ELSE print F
    RET
```

Appendix C - LC-3 Memory Layout

Label	Directive / Storage	Purpose
MAIN	.ORIG X3000	Program entry point.
STACK	.FILL xFDFF	Initial stack pointer value loaded into R6.
FIVE	.FILL #5	Constant used for five scores and division by five.
NEG5	.FILL #-5	Used for average division by repeated subtraction.
NEG10	.FILL #-10	Used by PRINT_NUM during division by ten.
HUNDRED	.FILL #100	Maximum score used when clamping input.
NEG100	.FILL #-100	Used to compare input value against 100.
NEG48	.FILL #-48	Used to convert ASCII digits into numeric values.
NEG9	.FILL #-9	Used to reject characters above digit 9.
ASCII0	.FILL x0030	Used to convert numeric digits into ASCII for output.

NEWLINE	.FILL x000A	Used to detect and print newline.
T90 / T80 / T70 / T60	.FILL #90 / #80 / #70 / #60	Grade thresholds.
LETTERA / LETTERB / LETTERC / LETTERD / LETTERF	.FILL x0041 / x0042 / x0043 / x0044 / x0046	ASCII letter grades.
SPTR	.FILL #0	Stores the current pointer into the SCORES array during input.
CTR	.FILL #0	Stores the input loop counter.
MINV	.FILL #0	Stores calculated minimum score.
MAXV	.FILL #0	Stores calculated maximum score.
AVGV	.FILL #0	Stores calculated integer average.
SCORES	.BLKW 5	Array storage for five test scores.
PROMPT	.STRINGZ "Enter Score: "	Input prompt printed before each score.
MSG_MIN	.STRINGZ "\nMinimum: "	Minimum output label.
MSG_MAX	.STRINGZ "\nMaximum: "	Maximum output label.
MSG_AVG	.STRINGZ "\nAverage: "	Average output label.
MSG_GRADE	.STRINGZ " Grade: "	Grade output label.

Appendix D - Subroutine Summary

Subroutine	Purpose	Important Details
READ_SCORE	Reads one score from the keyboard and returns the numeric value in R0.	Uses GETC, OUT, NEWLINE, NEG48, NEG9, NEG100, and HUNDRED. Ignores non-digits, ends on Enter, and clamps above 100.

CALC_STATS	Calculates the minimum, maximum, and integer average of the five stored scores.	Uses SCORES, MINV, MAXV, AVGV, FIVE, and NEG5. Uses R4 as the running sum.
PRINT_RESULTS	Prints the final results.	Prints Minimum, Maximum, and Average. Calls PRINT_NUM and PRINT_GRADE for each displayed result.
PRINT_NUM	Prints the integer value currently in R0.	Uses repeated subtraction by ten and stack storage to reverse digit order before printing.
PRINT_GRADE	Prints the letter grade for the value in R0.	Uses thresholds T90, T80, T70, and T60, then prints A, B, C, D, or F.